

# CMR 2024/25: YEAR WORK REPORT

IACOPO MOLES (10560273)

## 1. PARAMETERS

| Assigned |   |   | Tuned            |                  |       |               |
|----------|---|---|------------------|------------------|-------|---------------|
| $T_a$    | T | a | $K_{px}, K_{py}$ | $T_{ix}, T_{iy}$ | $T_s$ | $\varepsilon$ |
| 0.15     | 8 | 5 | 2.09             | 28.6             | 1/200 | 0.1           |

## 2. SYSTEM ANALYSIS

Our system is composed of two actuators with transfer function

$$A(s) = \frac{1}{s + T_a}$$

The effects of the actuators are then integrated by the mobile robot.

Let's suppose the robot is aligned with either axis and that we supply a velocity command with no command on the rotation. We can see this is just as an integrator in series to our actuator, so the transfer function of the robot is:

$$G_{x/y}(s) = \frac{A(s)}{s} = \frac{1}{s(s + T_a)}$$

The same can be said about our yaw when we supply only a rotation command.

$$G_{\theta}(s) = \frac{A(s)}{s} = \frac{1}{s(s + T_a)}$$

If these two internal dynamics are “decoupled enough” then we can just design our controller on the  $G_{x/y/\theta}(s)$

## 3. TUNING

### 3.1. Continuous Time.

With MATLAB we use the PID Tuning tool to design a suitable controller. To use it we need to specify the tuning parameters:

I settled on a response time of 1 second following this reasoning:

The “lap time” is 8 seconds, the “first quadrant time” (see red arrow) is 1/4 of that. Ideally I want the robot to be already aligned with the trajectory at the end of the “quasi-linear” section (see green arrow), so at around 1 second. For the behaviour I choose a 0.9 = Max Robustness to avoid oscillations

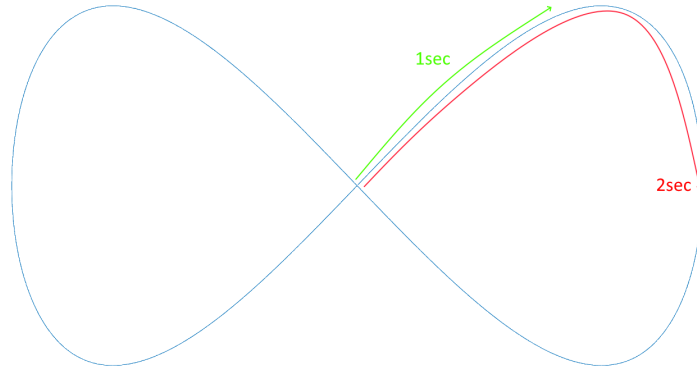


FIGURE 1. Time parametrization of the trajectory

*Instructions.*

In MATLAB:

- Make sure to have the “Control System Toolbox” installed
- Run the first code section in “PIDtune.mlx”, from line 1 to 43.

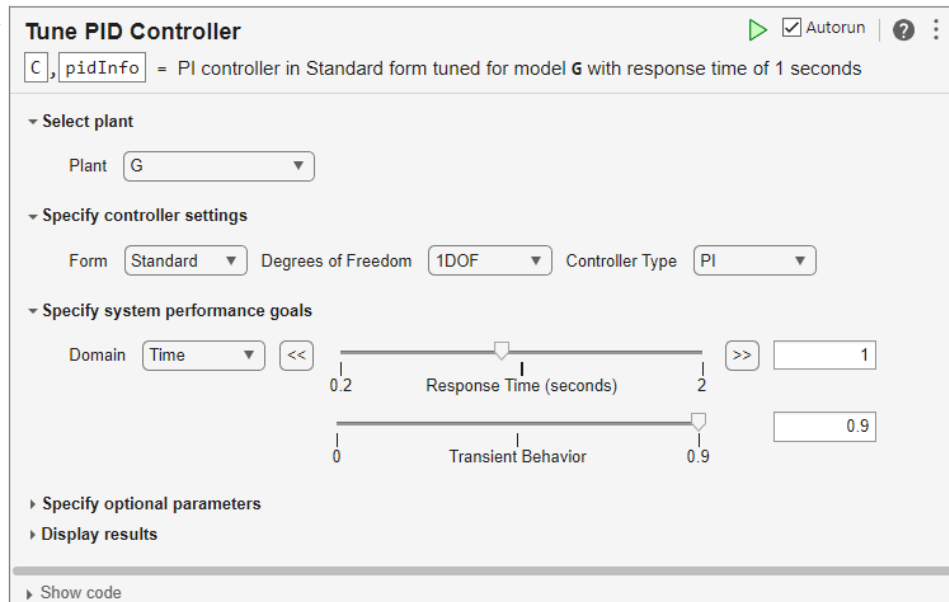


FIGURE 2. PID tuning interface

Output of the parameters proposed by MATLAB:

C =

$$K_p * \left( 1 + \frac{1}{T_i s} \right)$$

with  $K_p = 2.09$ ,  $T_i = 28.6$

Continuous-time PI controller in standard form

The feedback response of the PI controller and the robot has the following characteristics:

*Instructions.*

In MATLAB:

- Run the second code section in “PIDtune.mlx”, from line 44 to 52.

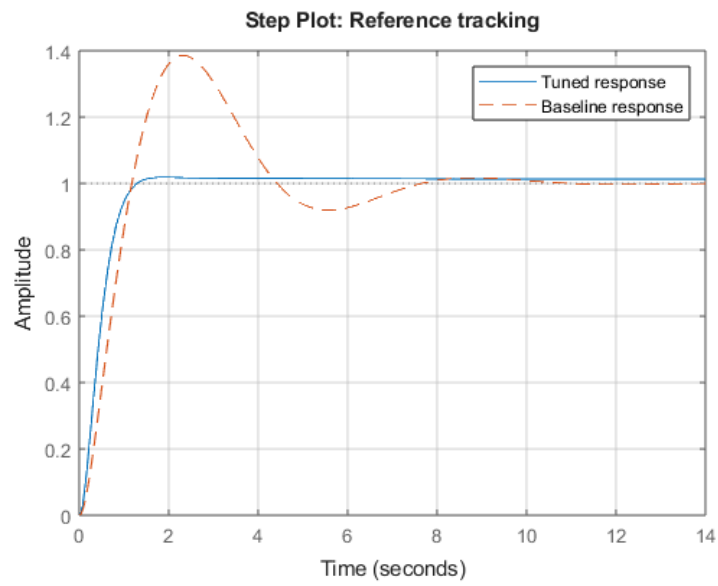


FIGURE 3. Tuned step response in continuous time compared to a naive  $K_p=1$ ,  $T_i=1$

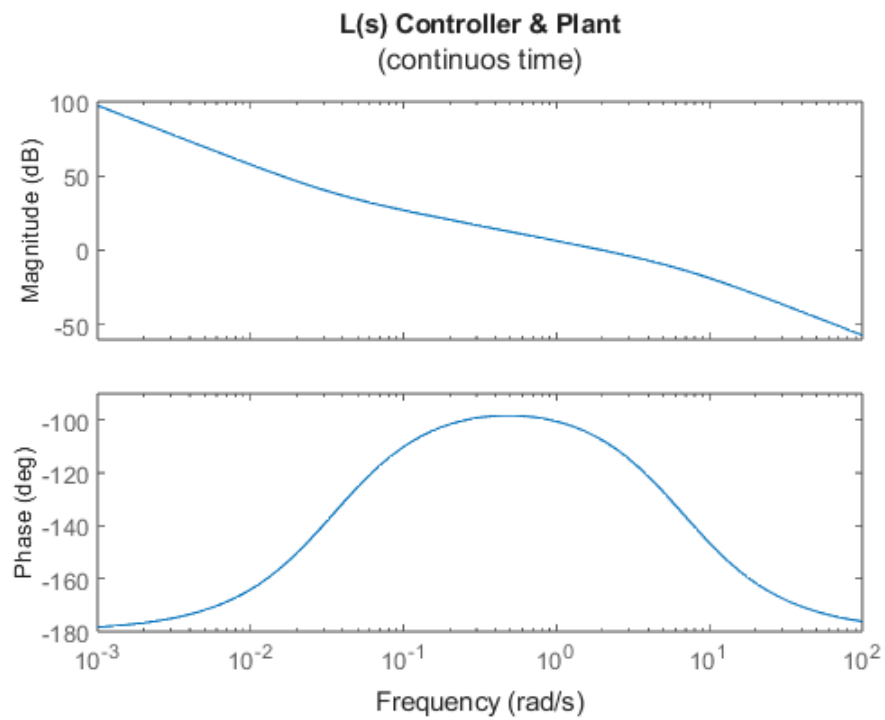


FIGURE 4. L(s) bode plot in continuous time

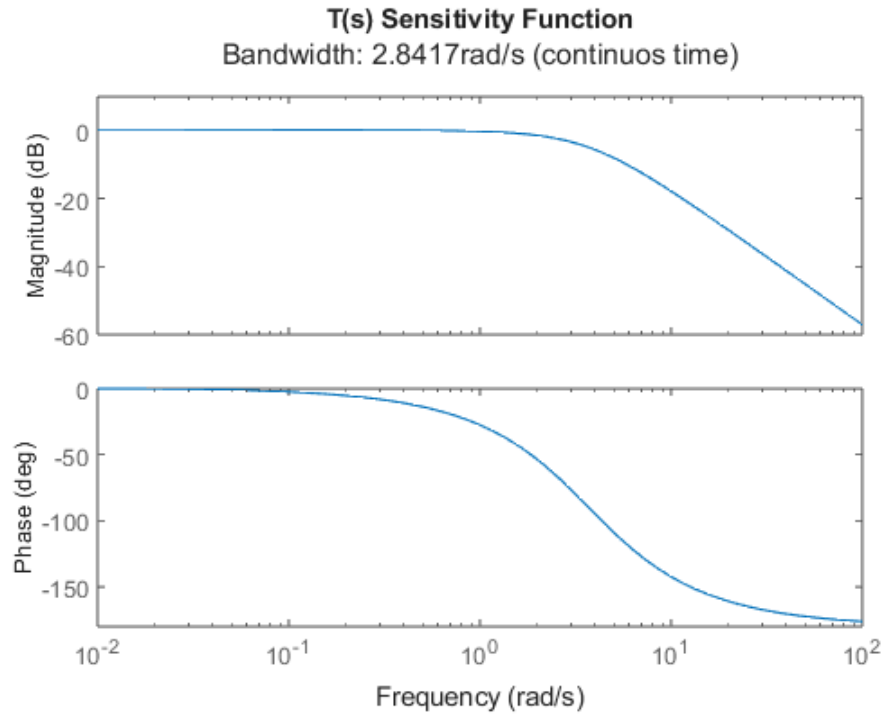


FIGURE 5.  $T(s)$  bode plot in continuous time

### 3.2. Model Validation.

I implemented a dynamic model in SIMULINK (see “tb\_kinematics.slx”) to see how the robot behaves in a “perfect” environment.

The model can be run in the standard way by just launching the file and making sure that the parameters are present in the workspace. Once validated that Everything works we can export them back onto MATLAB for further elaboration.

#### *Instructions.*

Nb: Opening SIMULINK is not necessary, the simulation is automatically launched by the MATLAB script with the appropriate parameters.

In MATLAB:

- Run the third code section in “PIDtune.mlx”, from line 52 to 86

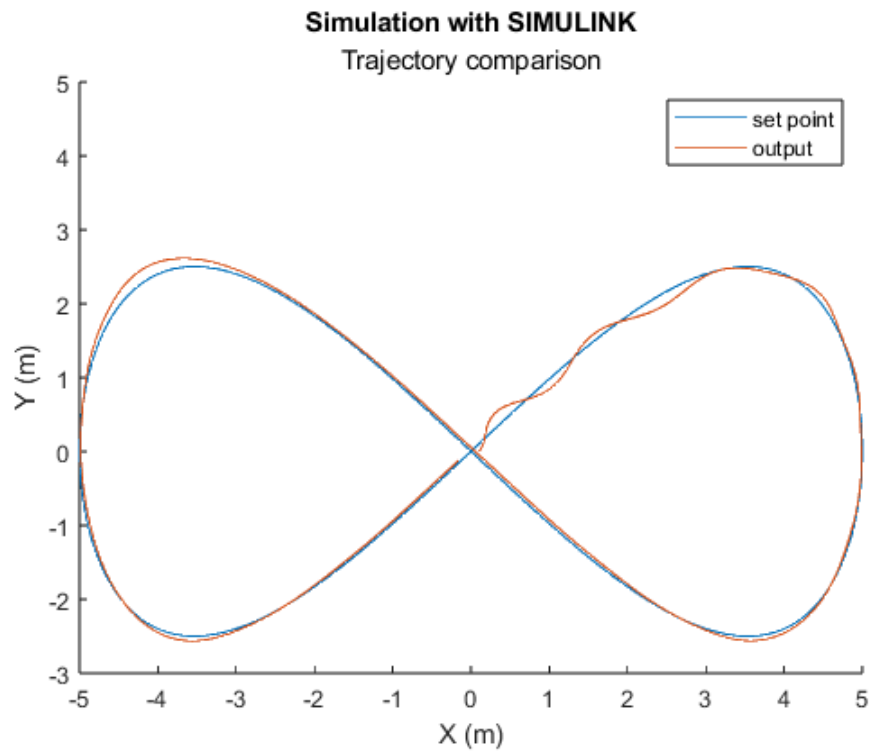


FIGURE 6. Plotted trajectory

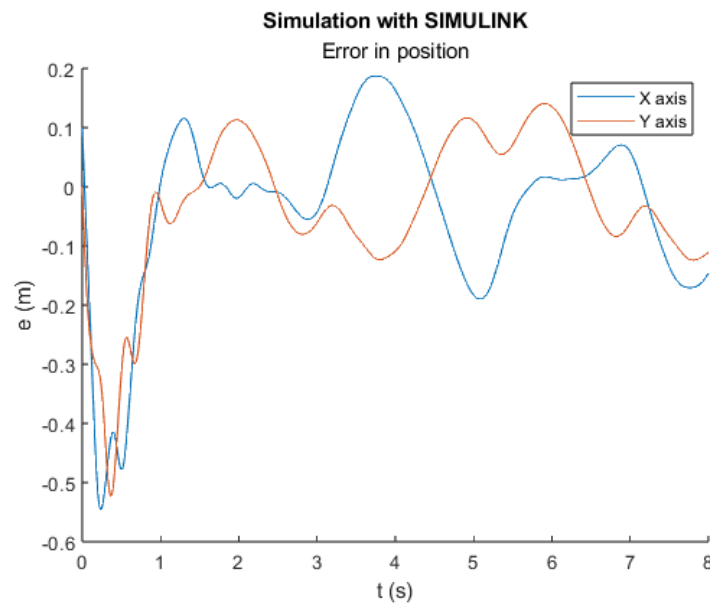


FIGURE 7. Error in the X,Y components for 1 lap time

From Figure 7 it's possible to see how at first the error goes to zero after about 1 second as designed. After that the absolute value of error oscillates between below 0.2m. This is due to a lag caused by our 2 poles in the robot plus the pole in the PI. Overall the zero in the PI is not enough to counteract their low pass filter effect.

### 3.3. Discrete Time.

In the previous simulation I picked a placeholder sampling frequency of 100Hz. From Figure 7 we can see at the start an error oscillation of around 0.25s for the X component (around 4Hz). For a proper tuning of the sampling frequency a thumb rule is to pick between 10 and 100 times the frequency of the fastest oscillations. In my case 50 times is a good trade-off.

*Instructions.*

In MATLAB:

- Run the fourth code section in “PIDtune.mlx”, from line 87 to 108

We can now discretize our system, obtaining the following response:

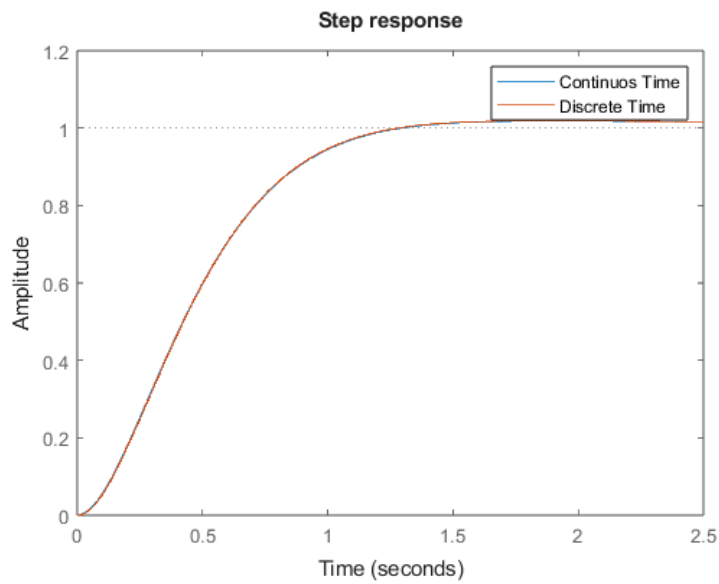
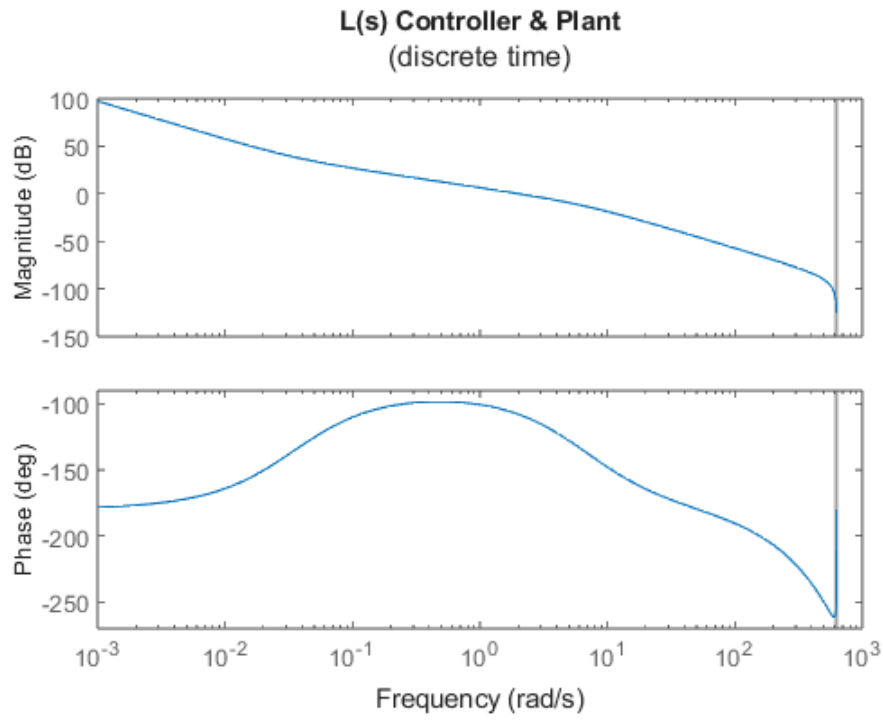
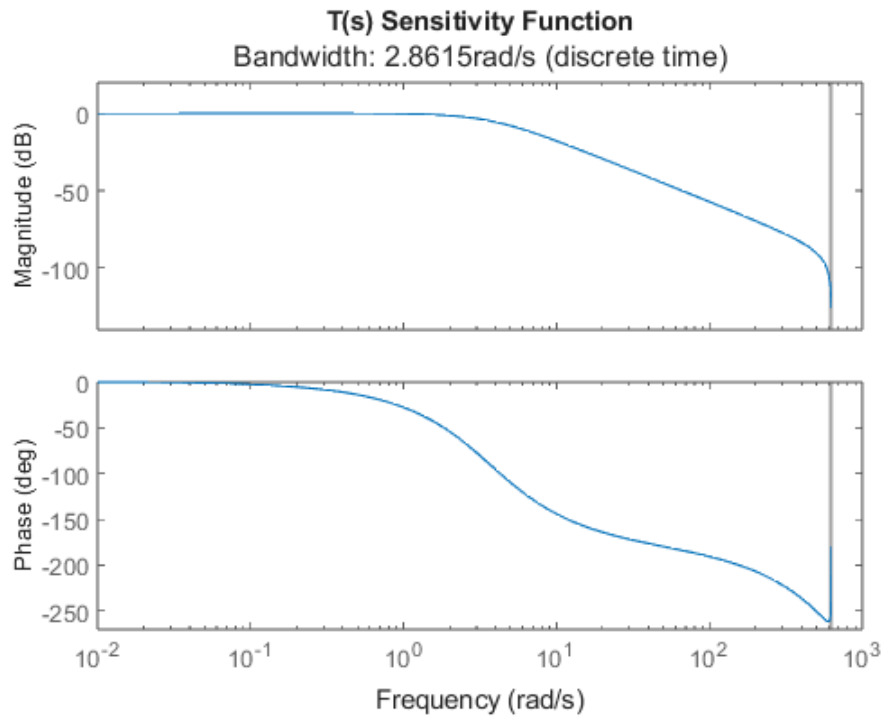


FIGURE 8. Step response at continuous time vs discrete time

FIGURE 9.  $L(s)$  bode plot in discrete timeFIGURE 10.  $T(s)$  bode plot in discrete time

From the bode plot we can see how the phase shape was preserved from the continuous-time design up until 10 rad/s ( $\approx 1.5\text{Hz}$ ).

## 4. SIMULATION WITH ROS

## 4.1. Simulator and open loop nodes.

The `turtlebot_simulator` package implements the following nodes:

1. Simulator Node:
  - Type: `turtlebot_simulator_node`
  - Published Topics:
    - `/tb_vel` [`geometry_msgs/TwistStamped`]
   
Velocity of our actuators.
    - `/state` [`geometry_msgs/PoseStamped`]
   
Position of our robot's COG.
    - `/clock` [`roscpp_msgs/Clock`]
   
Clock of the simulation.
  - Subscribed Topics:
    - `/cmd` [`geometry_msgs/TwistStamped`]
   
Input for the linear and angular velocities.
2. Open Loop Trajectory Node: (trajectory generation)
  - Type: `turtlebot_cmd_input`
  - Published Topics:<sup>1</sup>
    - `/cmd` [`geometry_msgs/TwistStamped`]

In this first package we can test the quality of our simulation by feeding the simulator node with arbitrary commands. I've chosen a simple combination:

$$v_{cmd}(t) := 0.1 \forall t$$

$$\omega_{cmd}(t) := \begin{cases} 0 & \text{if } 0 < t < 5 \\ 1.1 & \text{if } 5 \leq t < 7 \end{cases}$$

(Square wave of period 7 seconds, with the first 5 seconds off)

*Instructions.*

In the CATKIN workspace:

- Compile with `catkin_make`
- Start recording rosbag `record -a --duration 50`
- Launch the simulation with `roslaunch turtlebot_simulator simulator.launch`  
by default it will wait 1s before starting the trajectory generation to let rosbag subscribe correctly to the topic without missed messages.
- If PlotJuggler is installed it's possible to launch a realtime visualization of the simulation: `roslaunch turtlebot_simulator plotjugg.launch`  
The trajectory node will wait 1 second before starting to be able to launch the GUI correctly. Simply press OK when asked.

By default the simulation will terminate after a predetermined time (in the order of 60s)

In MATLAB:

- Run `bag_analysisT1.mlx`. By default it will look for a file named `t1.bag`. In my case it's already provided. Otherwise rename the file created by the rosbag step. It will automatically generate the appropriate plots.

From the plots we can see the quality of our setup. It mimics well enough the results from SIMULINK and acts as we would expect, albeit with a slight difference of around 0.01m in Figure 13.

<sup>1</sup>It also subscribes to the `/clock` topic but it's implicit so it's not reported for this and other nodes

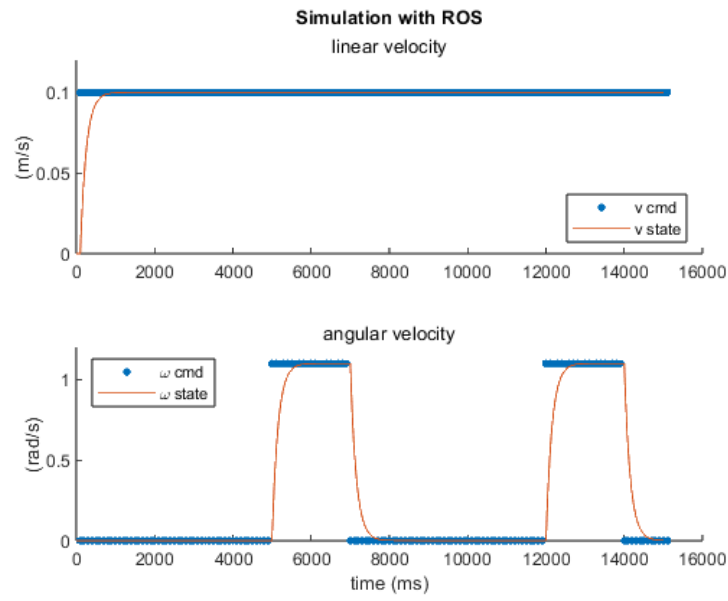


FIGURE 11. Actuator dynamics simulated with ROS

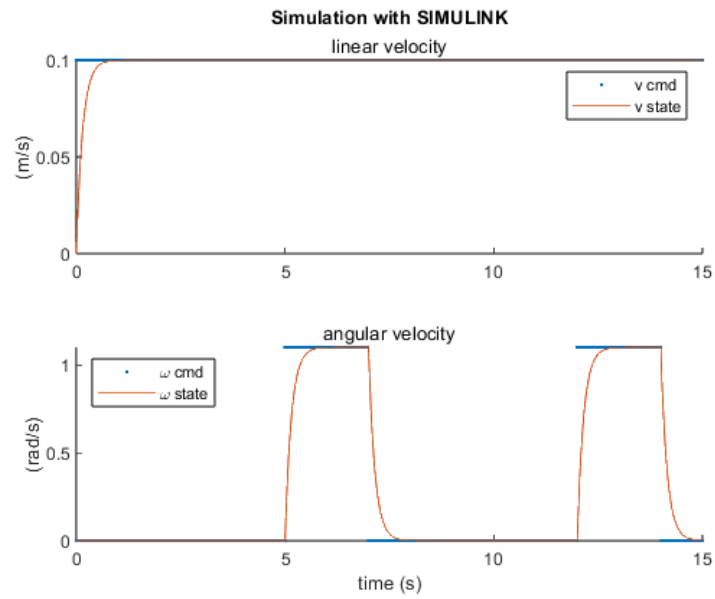


FIGURE 12. Actuator dynamics simulated with SIMULINK

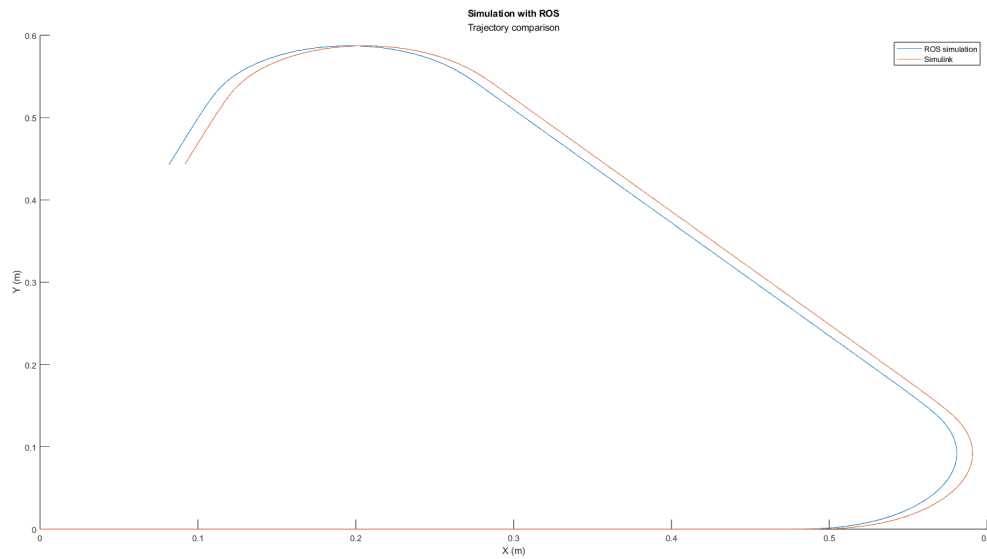


FIGURE 13. Trajectory generated by the test node compared to SIMULINK

#### 4.2. Trajectory Control Node.

I implemented the trajectory control in a ROS node. The node has the following characteristics:

- Type: `tb_ctrl_node`
- Published Topics:
  - `/cmd` [`geometry_msgs/TwistStamped`]
    - Speed and rotation commands calculated by the feedback law.
  - `/setpoint` [`geometry_msgs/PoseStamped`]
    - Desired XY coordinates of the COG.
  - `/state_p` [`geometry_msgs/PoseStamped`]
    - XY coordinates of the point P calculated from the state reported by the robot.
  - `/error` [`geometry_msgs/Vector3Stamped`]
    - Error in XY coordinates between the setpoint and P.
- Subscribed Topics:
  - `/state` [`geometry_msgs/PoseStamped`]
    - (see `turtlebot_simulator_node`)

##### *Instructions.*

In the CATKIN workspace:

- Compile with `catkin_make`
- Start recording rosbag `record -a --duration 20`
- Launch the simulation with `roslaunch turtlebot_traj_ctrl PID.launch`  
by default it will wait 1s before starting the trajectory generation to let rosbag subscribe correctly to the topic without missed messages.
- If PlotJuggler is installed it's possible to launch a realtime visualization of the simulation: `roslaunch turtlebot_traj_ctrl PID_PJ.launch`  
The trajectory node will wait 5 seconds before starting to be able to launch the GUI correctly.  
Simply press OK when asked.

By default the simulation will terminate after a predetermined time (in the order of 250s)

In MATLAB:

- Run `bag_analysisT2.mlx`. By default it will look for a file named `t2.bag`. In my case it's already provided. Otherwise rename the file created by the rosbag step. It will automatically generate the appropriate plots.

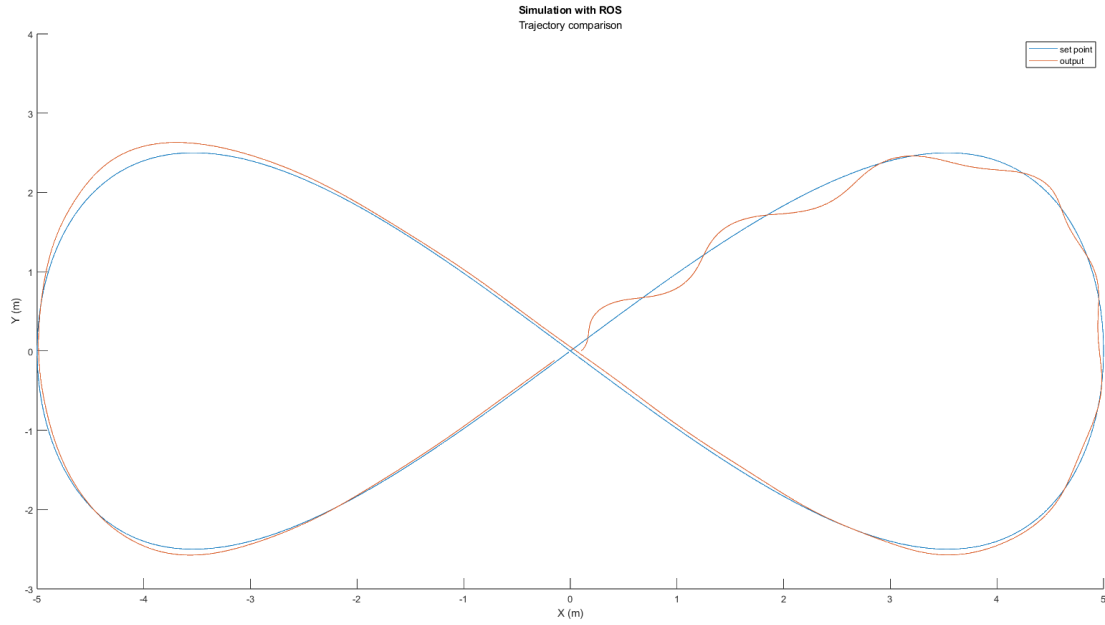


FIGURE 14. Requested and executed trajectory (point 2.a of the assignment)

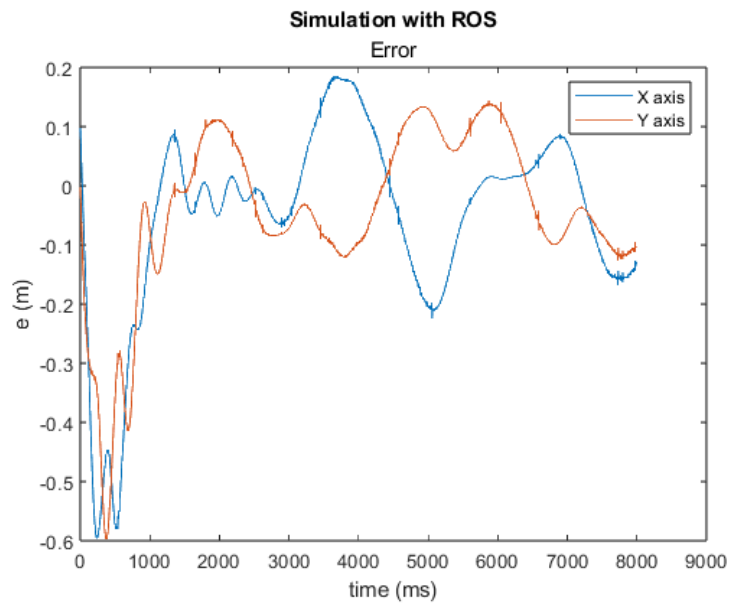


FIGURE 15. Error in the XY components (point 2.b of the assignment)

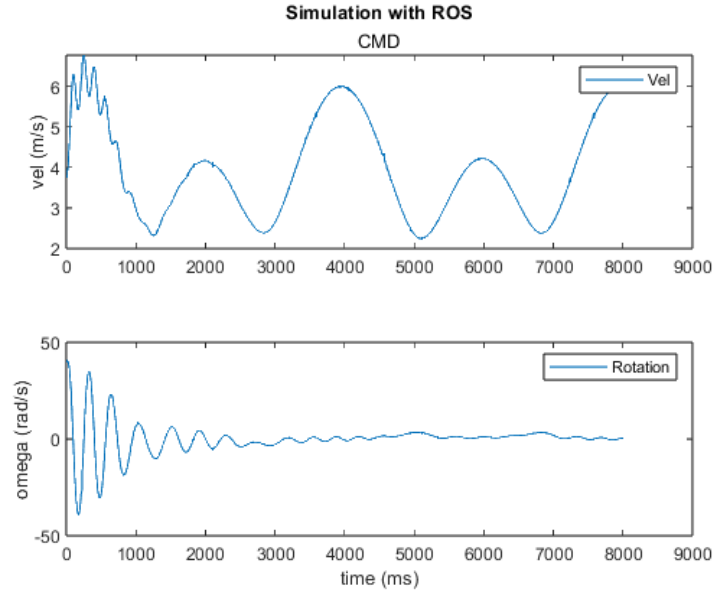


FIGURE 16. Control signals generated by the controller (point 2.c of the assignment)

We can see how Figure 7 and Figure 15 follow a similar evolution. Same for Figure 14 and Figure 6.

From Figure 16 we see how the effort requested to the linear velocity actuator has some oscillations at the start but it remains well behaved overall, not too dissimilar to the steady state. For the steering command we have instead a big difference (more than one of order of magnitude bigger) between the control signals at the start and the control signals at the steady state condition. This is caused by our strict settling time requirements and could be improved depending on the trade-off we want to make between controller effort and convergence speed.